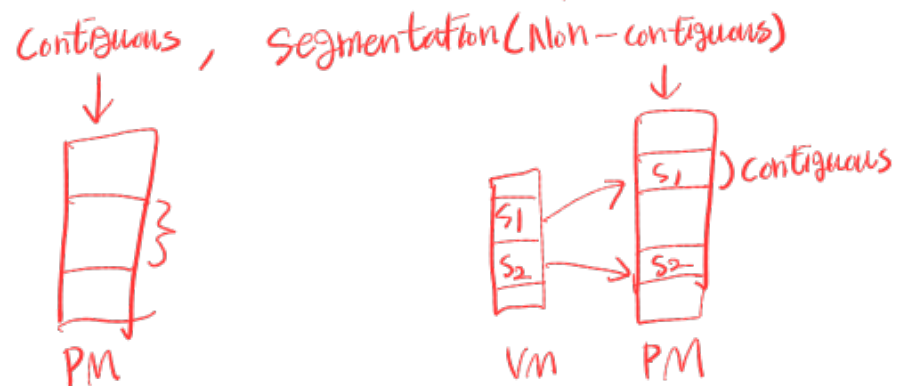


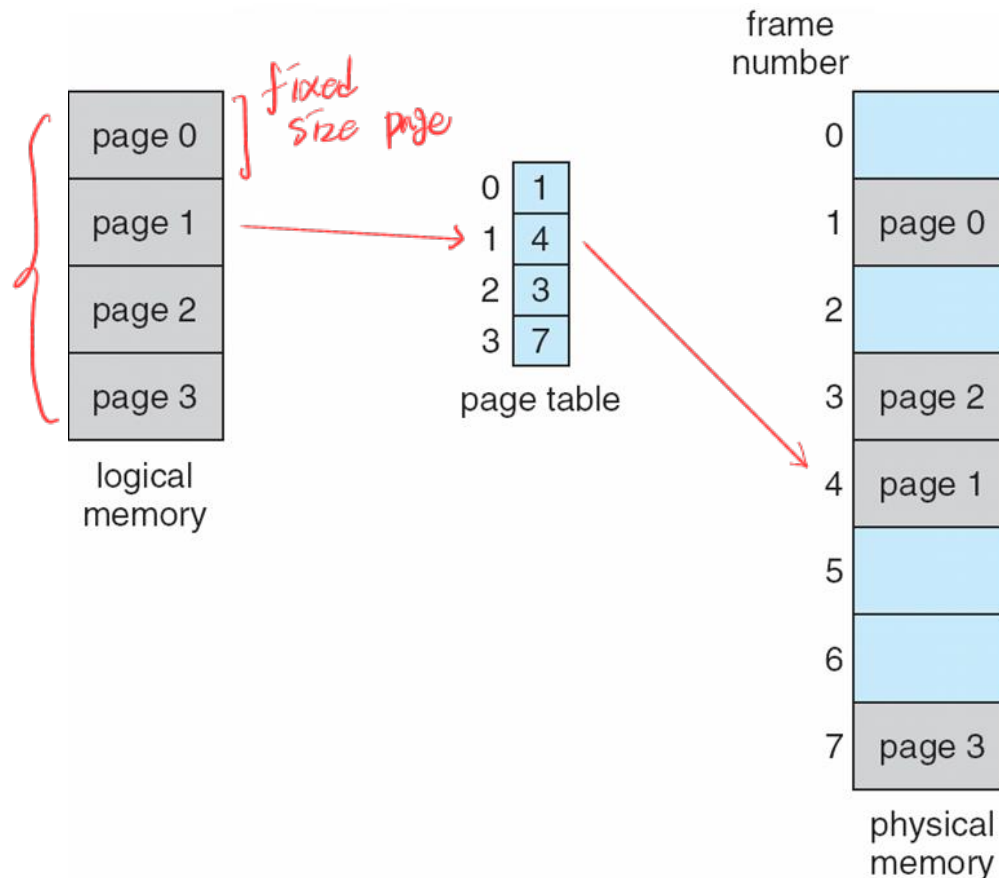
Paging

Chap 18, 19, 20



Paging

- Memory management scheme that permits the physical address space of a process to be noncontiguous
- Implemented by closely integrating the hardware and operating system

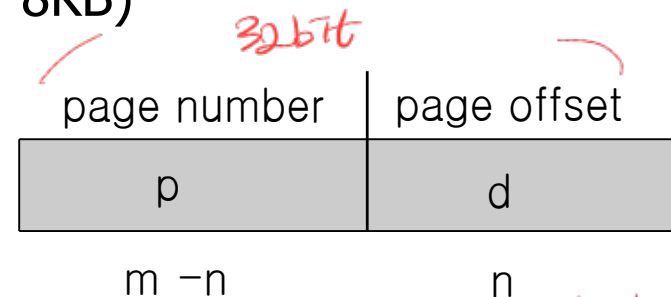


Paging

- **Basic method**

- Frames (page frames)
 - Breaking physical memory into fixed-size blocks
- Pages
 - Breaking logical memory into blocks of the same size
 - Page size
 - Typically power of 2
 - 512B ~ 16MB (typically 4KB ~ 8KB)
- Logical address
 - $v = (p, d)$
 - p: page number
 - d: page offset (displacement)

virtual address



4KB offset = 12bit

- logical address space 2^m $2^{12} = 4096$
- page size 2^n

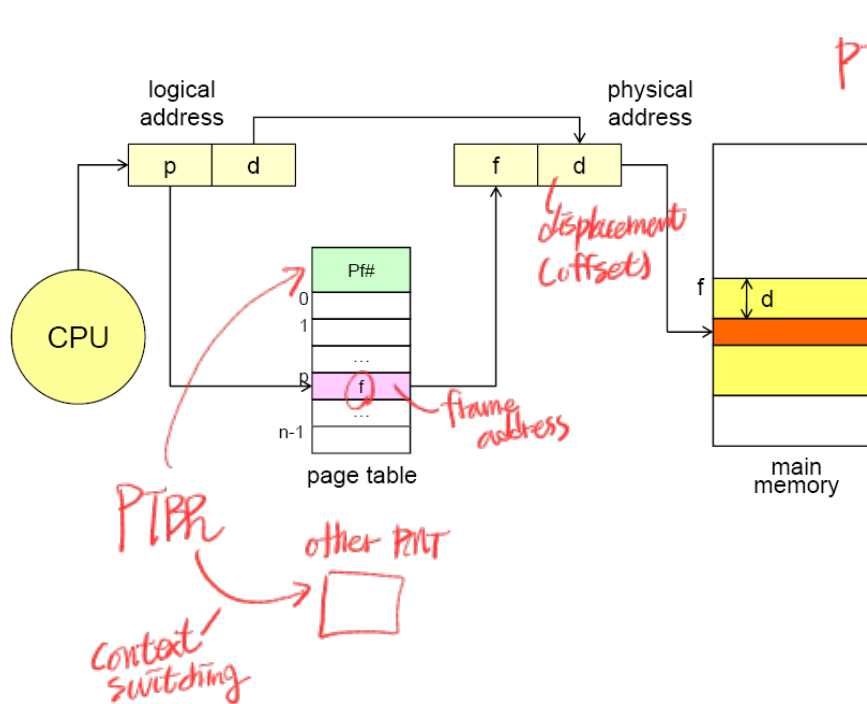
Paging

- **Basic method**
 - Address mapping
 - Logical address → physical address
 - Hidden from the user and controlled by the operating system
 - Uses page map table (PMT)
 - A page table for each process
 - PMT resides in kernel space of main memory

Paging

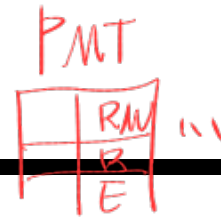
• Basic method

- Paging hardware and address mapping
- Page-table base register (PTBR) points to the page table
- Page-table length register (PTLR) indicates size of the page table



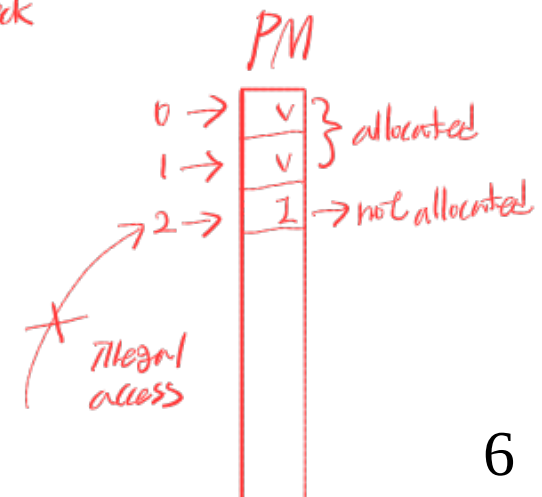
```
1 // Extract the VPN from the virtual address
2 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4 // Form the address of the page-table entry (PTE)
5 PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7 // Fetch the PTE
8 PTE = AccessMemory(PTEAddr)
9
10 // Check if process can access the page
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Access is OK: form physical address and fetch it
17     offset = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

Memory Protection

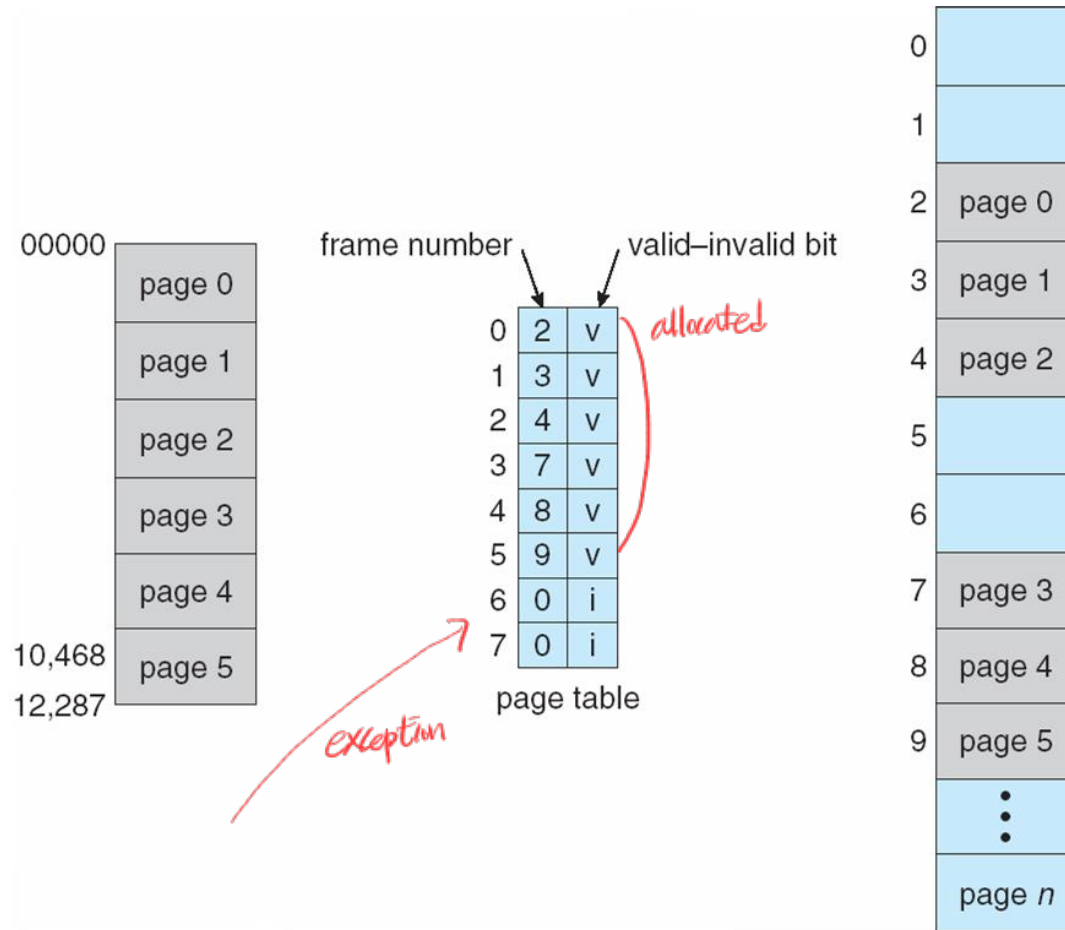


- **Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed**
 - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid bit attached to each entry in the page table:**
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use **page-table length register (PTLR)**
- **Any violations result in a trap to the kernel**

hardware check
exception



Valid (v) or Invalid (i) Bit In A Page Table



Page-table entry (PTE)

• Valid bit

- indicate whether the particular translation is valid
- All the unused space will be marked invalid, and if the process tries to access such memory, it will generate a trap → segmentation fault
- crucial for supporting a sparse address space
- remove the need to allocate physical frames for unallocated pages and thus save a great deal of memory.

• Protection bit (R/W, U/S)

• Present bit (P)

- Indicates whether this page is in physical memory or on disk

• Dirty bit (D)

- Indicates whether the page has been modified

• Reference bit (accessed bit) (A)

- used to track whether a page has been accessed

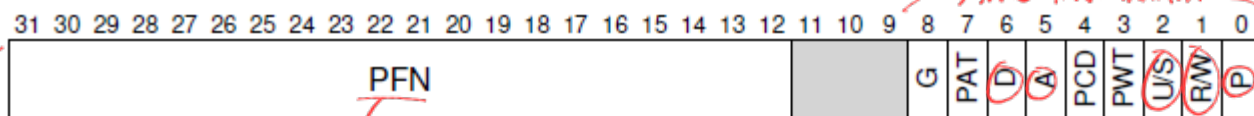
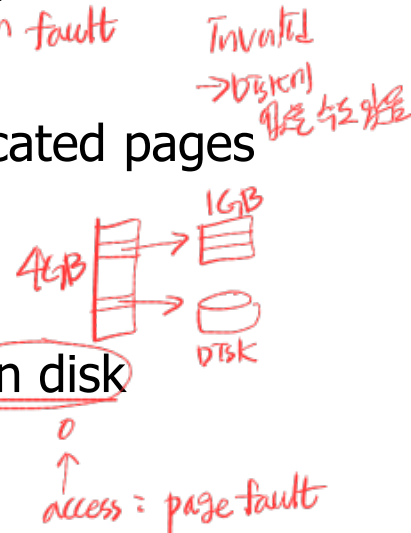
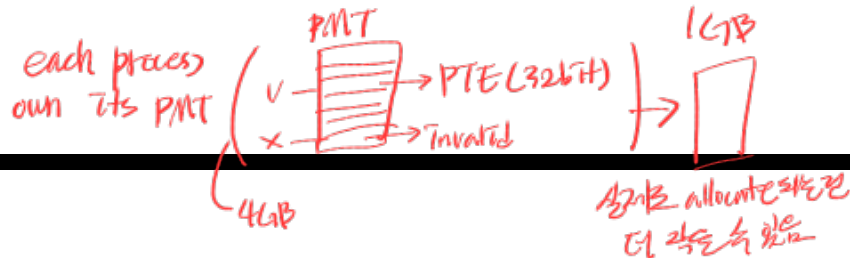


Figure 18.5: An x86 Page Table Entry (PTE)



No valid bit?

p=1 present, valid
p=0 present, invalid
Invalid
page 24B
page 24B X

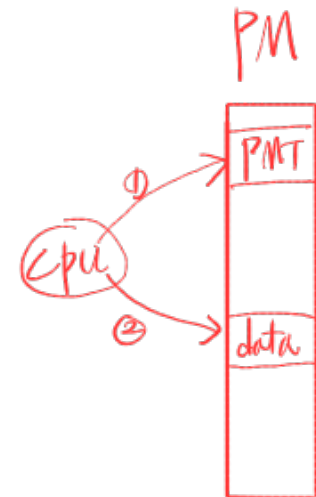
Page Table Location

- **Direct mapping**

- Doubles the number of memory accesses during execution of the program - One for the page table and one for the data/instruction
- Performance degradation ↓

- **TLB (Translation Look-aside Buffer)**

- PMT: maintained in kernel space of the memory
- Subset of PMT entries: maintained in small TLB
 - Entries of the recently used pages
 - dedicated registers or cache memories

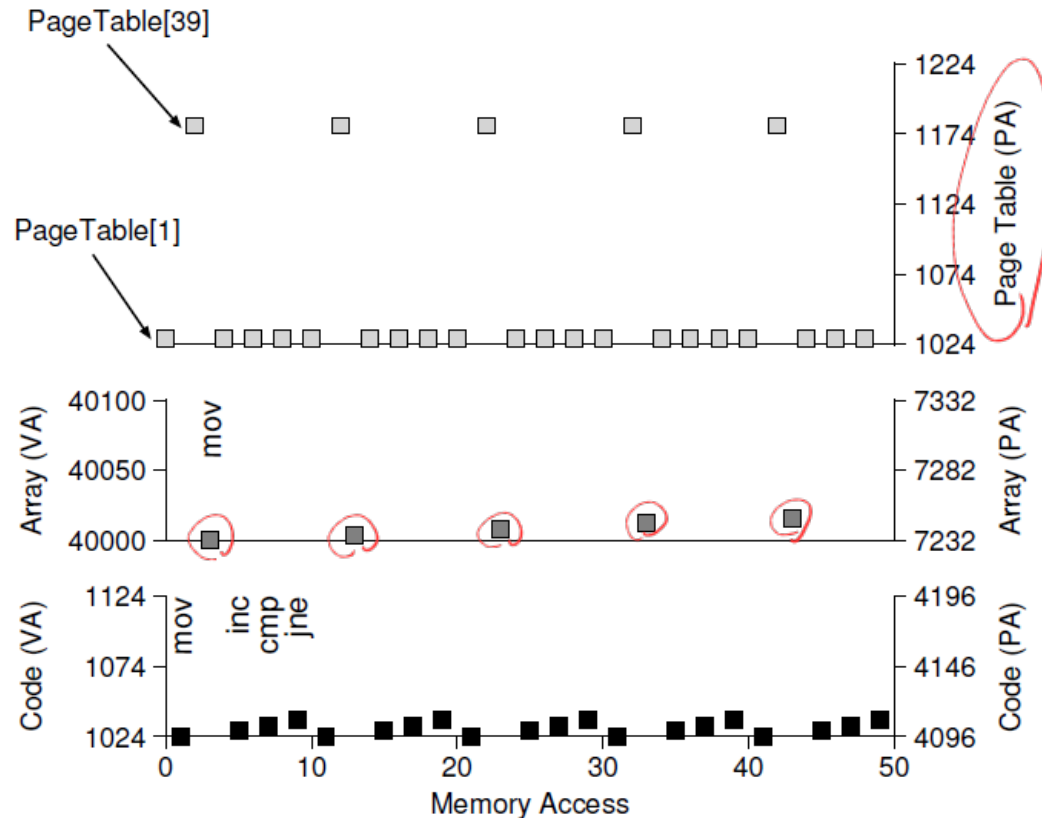


Memory Trace of Direct mapping

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

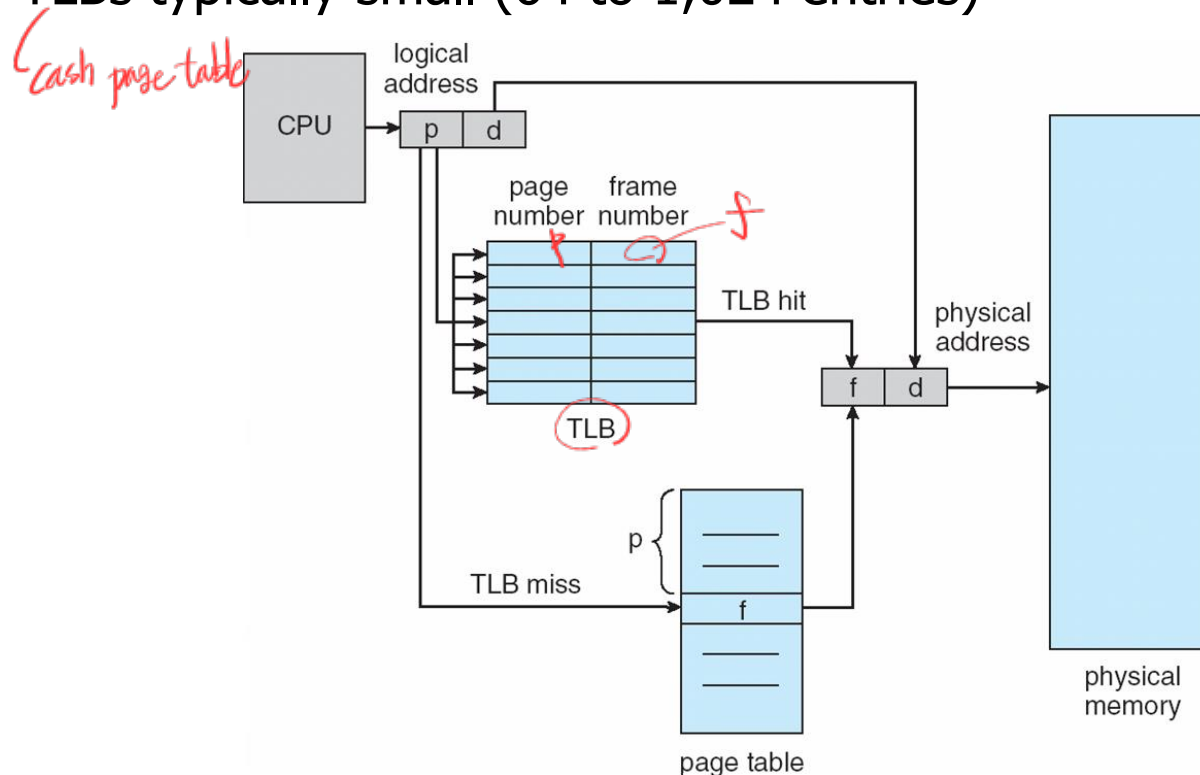
↓ compile

```
1024 movl $0x0,(%edi,%eax,4)  
1028 incl %eax  
1032 cmpl $0x03e8,%eax  
1036 jne 0x1024
```



TLB (Translation Look-aside Buffer)

- **Associative memory in MMU – parallel search**
- **Address translation (p, d)**
 - If p is in associative register, get frame # out
 - Otherwise get frame # from page table in memory *TLB miss*
 - TLBs typically small (64 to 1,024 entries)



Effective Access Time

- TLB hit ratio $\alpha = 80\%$
- TLB search: 20 ns
- memory access: 100 ns
- $EAT = 0.80 \times (20 + 100) + 0.20 \times (20 + 100 + 100) = 140\text{ns}$
 - 40% slowdown in memory access time

$$\begin{array}{l} (20 \times 0.8 = 16) \\ (20 \times 0.2 = 4) \end{array} \Bigg) 100 = 140$$

Who Handles The TLB Miss?

1111-0000..

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN) TLB
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE)) PMT의 PTE 32
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```



software-managed TLB (RISC)

On a TLB miss, hardware simply raises an exception

→ **OS raises exception** and **RaiseException(TLB_MISS)**

Need to **replace** an old one (LRU, Random)

hardware-managed TLB (CISC)

Hardware has to know exactly where the page tables are located in memory (via **PTBR**), as well as their exact format; on a miss. Hardware “walks” the page table, find the correct page-table entry and extract the desired translation, update the TLB with the translation, and retry the instruction

Software-managed TLB

- When returning from a TLB miss-handling trap, the hardware must resume execution at the instruction that **caused** the trap
 - This retry thus lets the instruction run again, this time resulting in a TLB hit.
- When running the TLB miss-handling code^{s/w}, the OS needs to be extra careful not to cause an infinite chain of TLB misses to occur.
 - Keep TLB miss handlers in physical memory (where they are unmapped and not subject to address translation),
 - Or reserve some entries in the TLB for permanently-valid translations (wired translation) *critical software code*
- Flexible *HW*
 - OS can use any data structure it wants to implement the page table, without necessitating hardware change.
- Simple *HW*
 - hardware doesn't have to do much on a miss; it raises an exception, and the OS TLB miss handler does the rest.

TLB Valid Bit $\neq$ Page Table Valid Bit

- **TLB entry also has several status bits**
 - valid, protection, dirty, ...
- **Invalid in PTE**
 - The page has not been allocated by the process
 - The access to an invalid page causes a trap to the OS
- **Invalid in TLB**
 - a TLB entry has not a valid translation within it
 - When a system boots, a common initial state for each TLB entry is to be set to invalid, because no address translations are yet cached there.
 - Context switch will set all TLB entries to invalid

TLB Issue: Context Switches

- TLB contains virtual-to-physical translations that are only valid for the currently running process
- When switching from one process to another, the hardware or OS (or both) must be careful to ensure that the about-to-be-run process does not accidentally use translations from some previously run process.
- Solutions
 - Flush the TLB on context switches, sets all valid bits to 0
 - incur TLB misses when a new process runs → high cost
 - **Address space identifier (ASID) $\approx$ (PID)**
 - enable sharing of the TLB across context switches

	VPN	PFN	valid	prot
P1 →	10	100	1	rwX
	—	—	0	—
P2 →	10	170	1	rwX
	—	—	0	—

	VPN	PFN	valid	prot	ASID
P1	10	100	1	rwX	1
	—	—	0	—	—
P2	10	170	1	rwX	2
	—	—	0	—	—

process 구분

Shared Pages

- **Shared code**

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

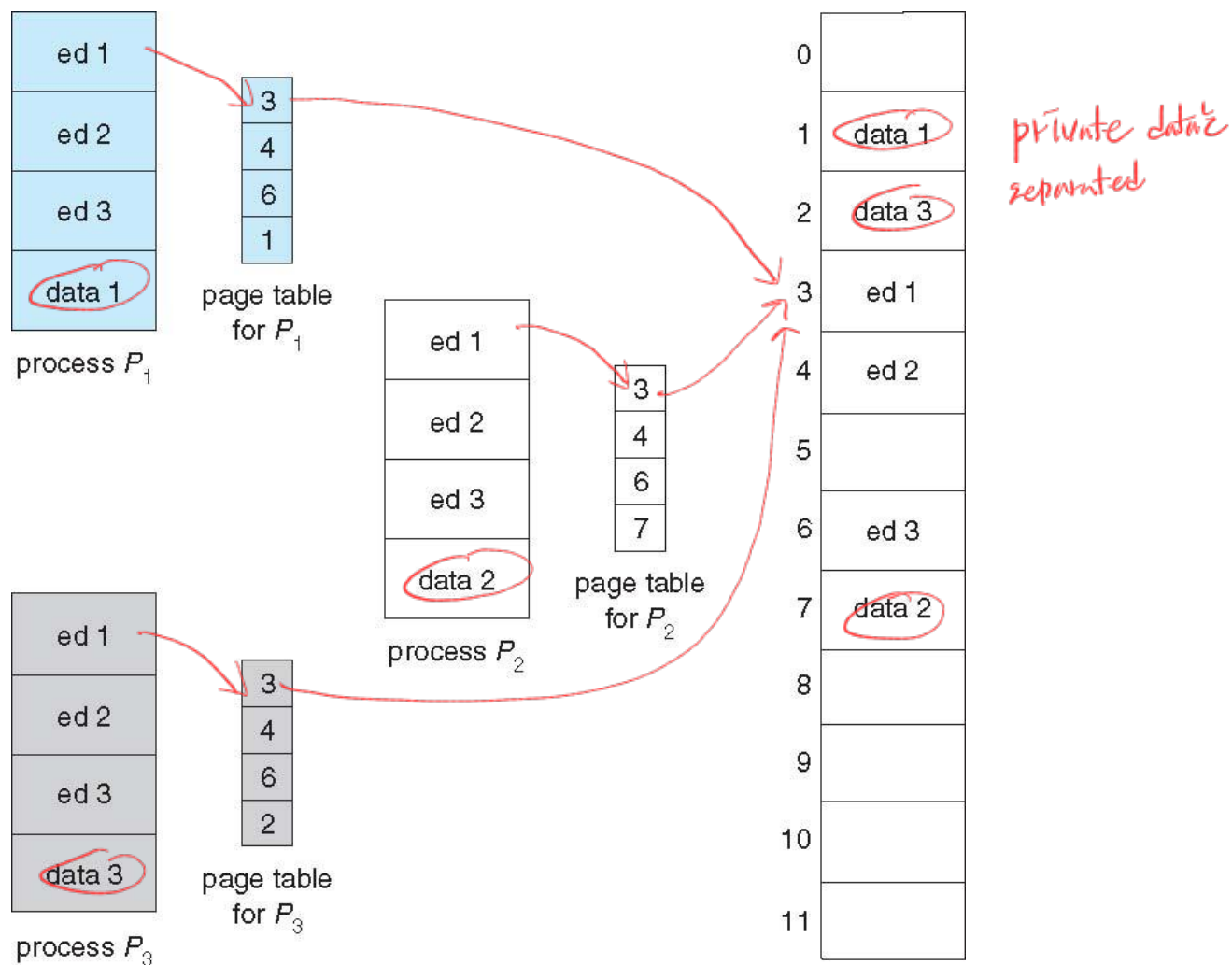
- **Private code and data**

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space

	VPN	PFN	valid	prot	ASID
P_1	10	101	1	r-x	1
	—	—	0	—	—
P_2	50	101	1	r-x	2
	—	—	0	—	—

Share

Shared Pages Example



A Real TLB Entry

- MIPS R4000, software-managed TLB
- 32-bit address space with 4KB pages.
 - 19-bit VPN and 12-bit offset $19+12=31$
 - user addresses will only come from half the address space (the rest reserved for the kernel)
- 24-bit physical frame number (PFN), up to 64GB (2^{24} 4KB pages).
- Global bit (G): globally-shared among processes (ASID is ignored)
- ASID: to distinguish between address spaces
- Coherence (C): determine how a page is cached by the hardware
- Dirty bit (D)
- Valid bit (V): valid translation
- 32 or 64 TLB entries
 - a few are reserved for the OS.
- Privileged instructions for TLB management
 - TLBP (probe), TLBR (read), TLBWI (write invalidate), TLBWR (write replacement)

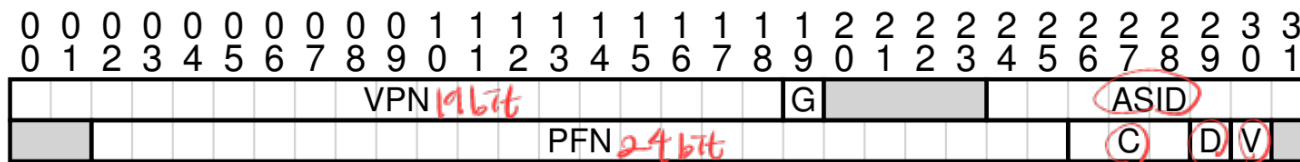
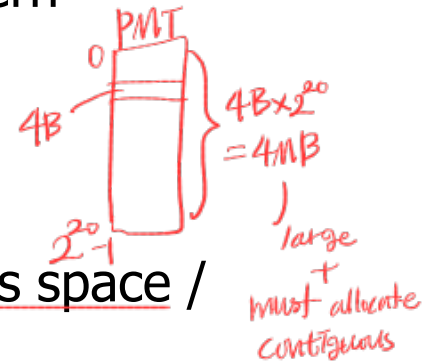


Figure 19.4: A MIPS TLB Entry

Paging: Smaller Tables

- **Memory structures for paging can get huge using straight-forward methods**

- Consider a 32-bit logical address space as on modern computers
- Page size of 4 KB (2^{12})
- Page table would have 1 million entries ($2^{32} / 2^{12}$)
- If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone



- That amount of memory used to cost a lot
- Don't want to allocate that contiguously in main memory

- **Bigger Page**
- **Hybrid Approach: Paging and Segment**
- **Multi-level Page Table (Hierarchical Paging)**
- **Hashed Page Table**
- **Inverted Page Table**

해결방법 5

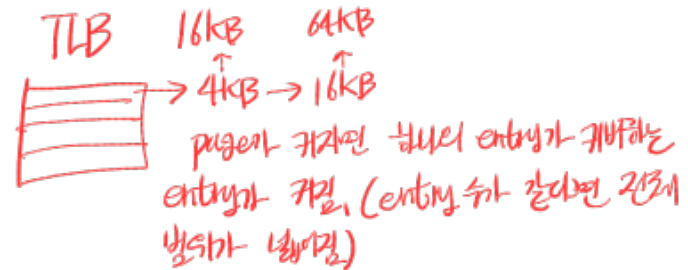
Simple Solution: Bigger Pages

- **16KB pages (2^{14})**

- $(2^{32} / 2^{14}) * 4B = 2^{20}B = 1MB$

- Large TLB coverage

- **Internal fragmentation**



- Most systems use relatively small page sizes

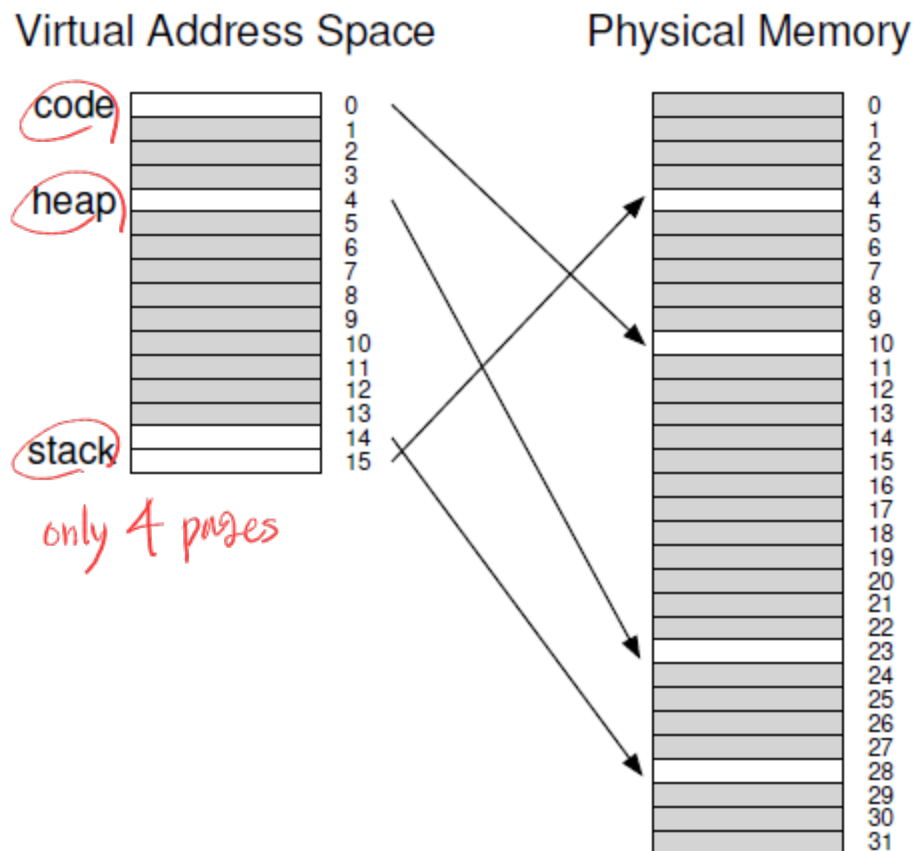
- 4KB (x86) or 8KB (SPARCV9)

- Many architectures now support multiple page sizes

- MIPS, SPARC, x86-64

4KB, 1KB, 64KB, 1MB

Sparse Page Table



A 16KB Address Space With 1KB Pages

Most of the page table is unused,
full of invalid entries

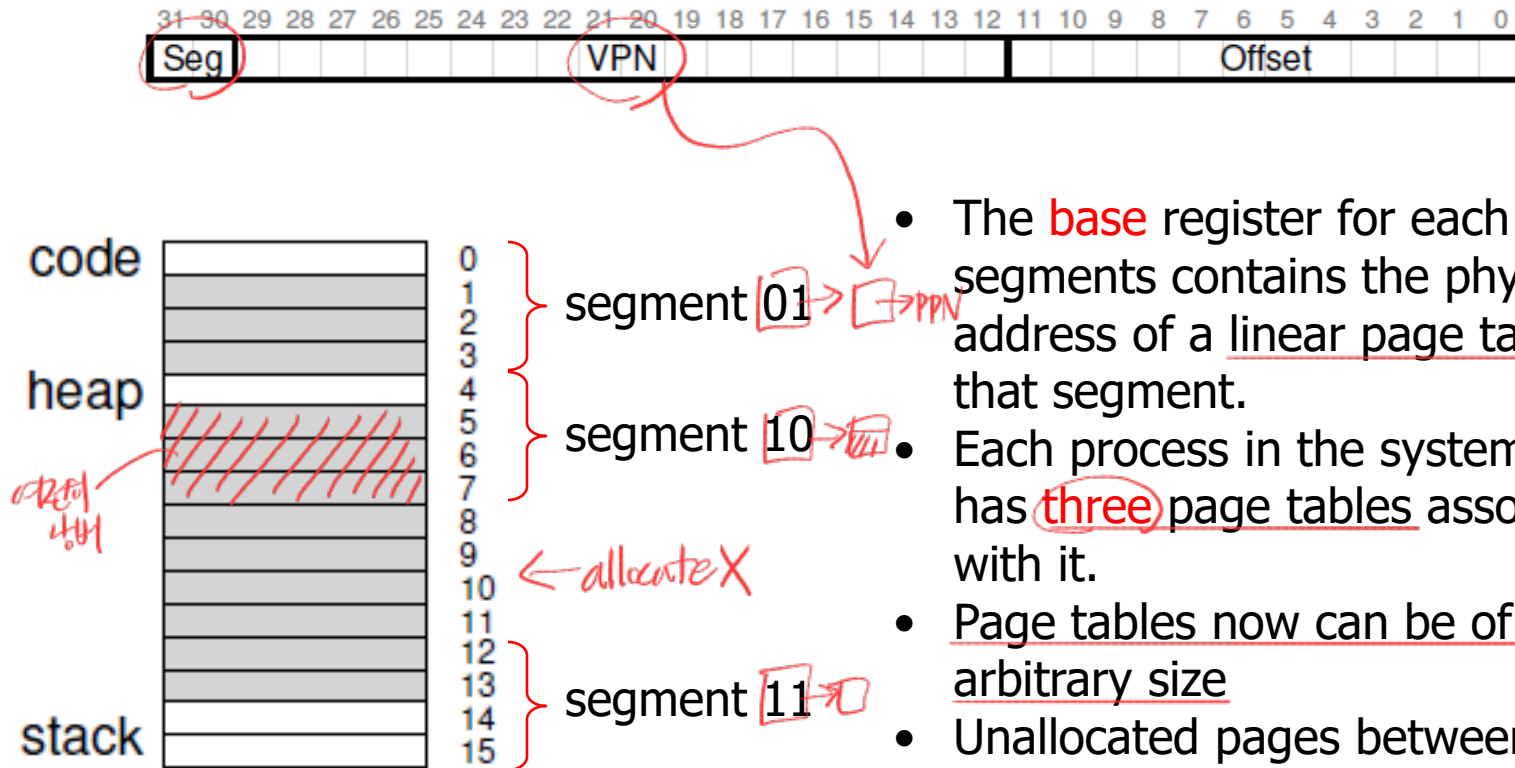
PMT = 16 entries

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
23	1	rw-	1	1
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
28	1	rw-	1	1
4	1	rw-	1	1

Wasted

Page Table

Hybrid Approach: Paging and Segments

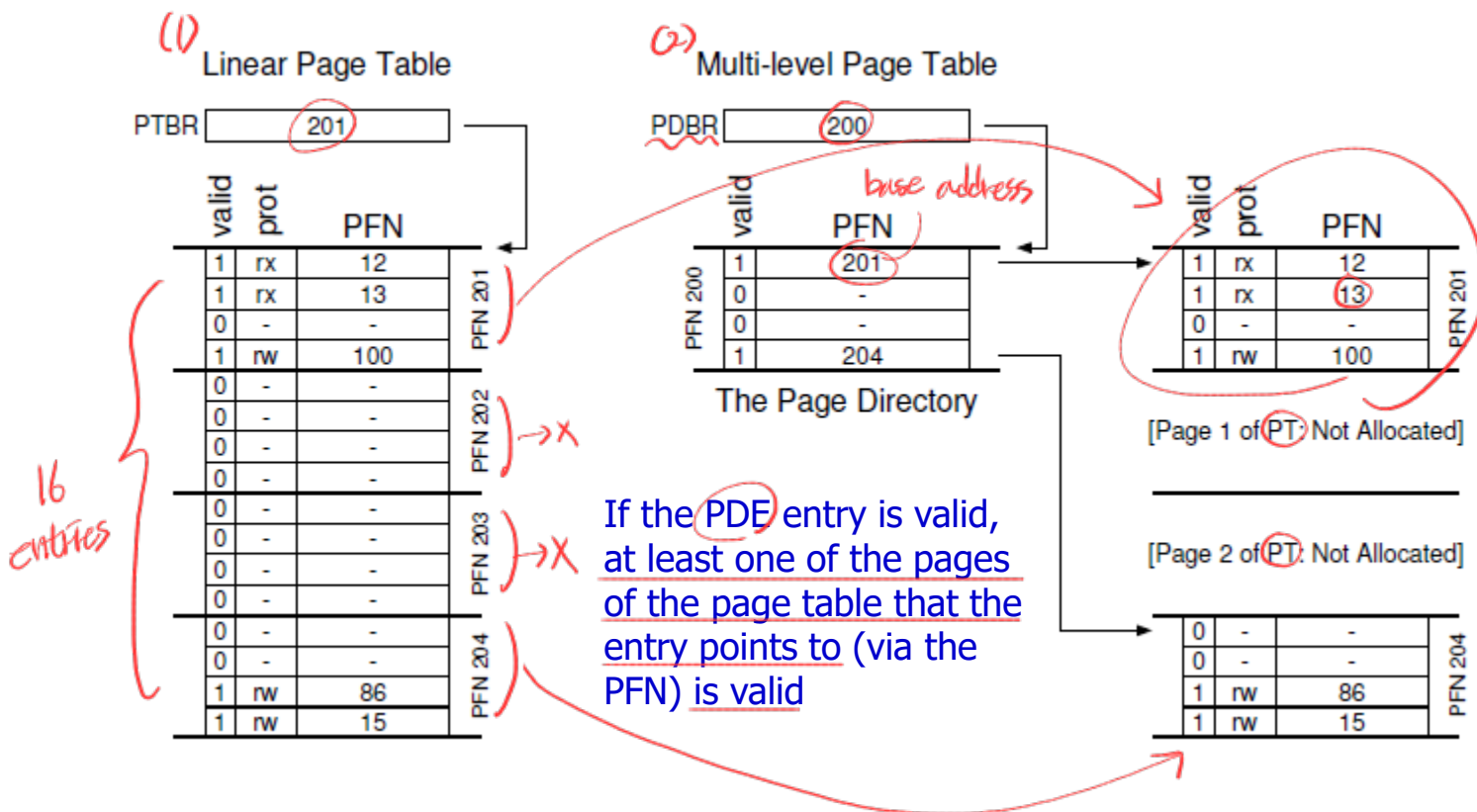


- The **base** register for each of these segments contains the physical address of a linear page table for that segment.
- Each process in the system now has three page tables associated with it.
- Page tables now can be of arbitrary size
- Unallocated pages between the stack and the heap no longer take up space in a page table

- Not flexible: if we have a large but sparsely-used heap, we can still end up with a lot of page table waste
 - External fragmentation
- segment 11

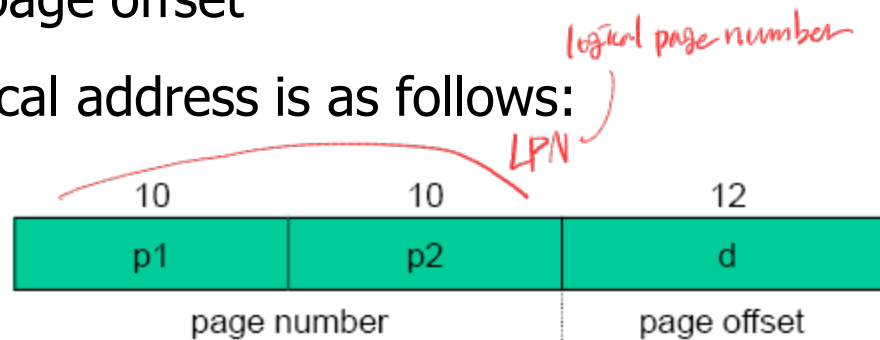
Multi-level (Hierarchical) Page Tables

- Break up the logical address space into multiple page tables
- Multi-level paging, in which the page table is also paged
- A simple technique is a two-level page table



Two-Level Paging Example

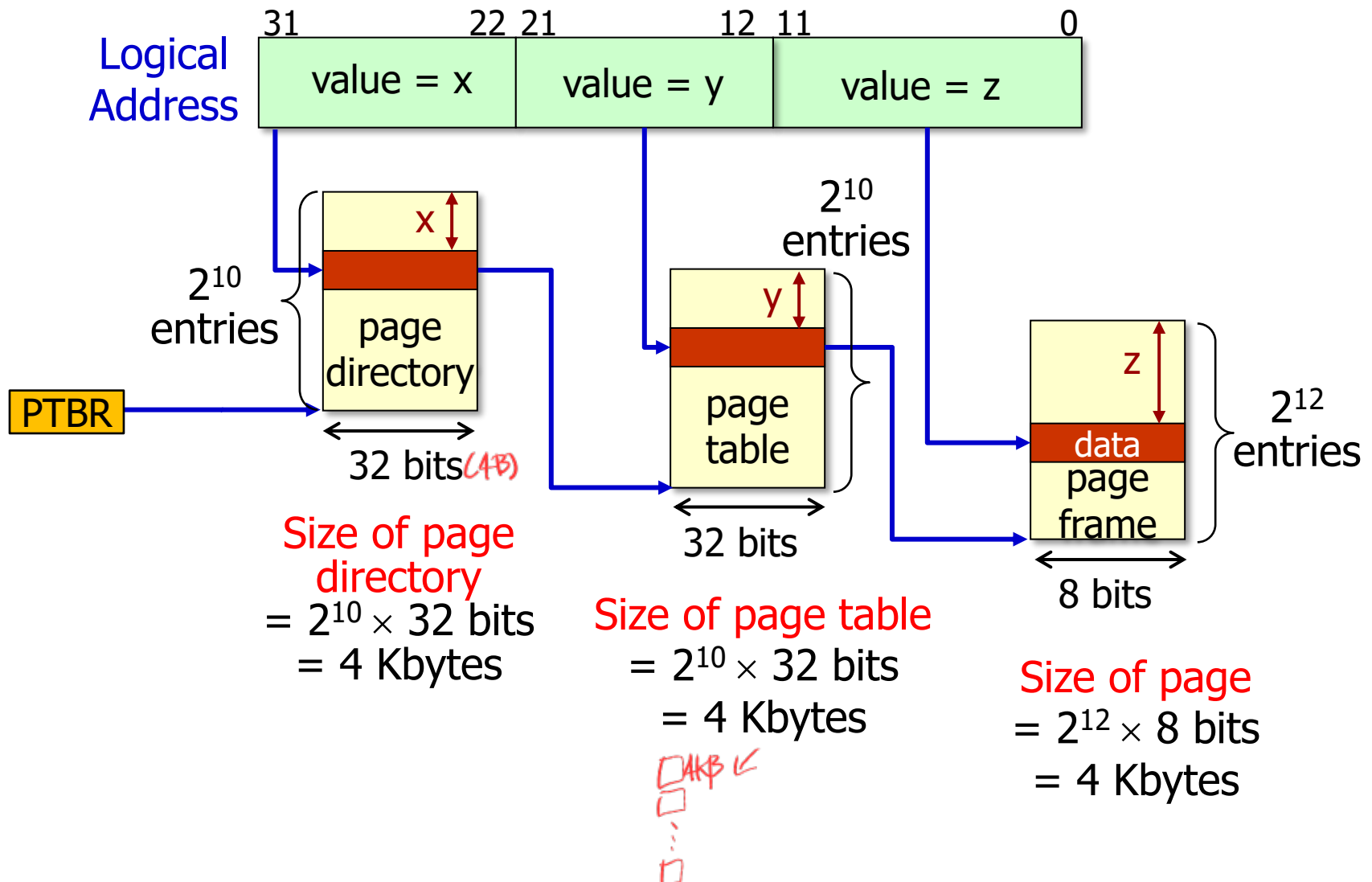
- A logical address (on 32-bit machine with 4K page size) is divided into:
 - a page number consisting of 20 bits
 - a page offset consisting of 12 bits (4KB page)
- Since the page table is paged, the page number is further divided into:
 - a 10-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:



- where p_1 is an index into the page directory, and p_2 is the displacement within the page of the page table

Address-Translation Scheme

two-level



Two-Level Page Table

• Small (typical) system

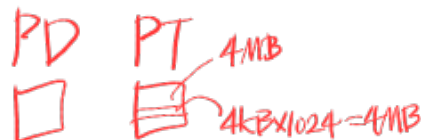
– One page table might be enough

- Page directory size + Page table size = 8 Kbytes of memory would suffice for virtual memory management

– How much *physical* memory could *this one page table* handle?

- Number of page tables * Number of page table entries * Page size =

$$1 * 2^{10} * 2^{12} \text{ bytes} = 4 \text{ Mbytes}$$



• Large system

– You might need the maximum number of page tables

- Max number of page tables * Page table size = 2^{10} directory entries * 2^{12} bytes = 2^{22} bytes = 4 Mbytes of memory would be needed for virtual memory management

– How much *physical* memory could *these 2^{10} page tables* handle?

- Number of page tables * Number of page table entries * Page size =

$$2^{10} * 2^{10} * 2^{12} \text{ bytes} = 4 \text{ Gbytes}$$

$$4 \text{ MB} \times 1024 = 4 \text{ GB}$$

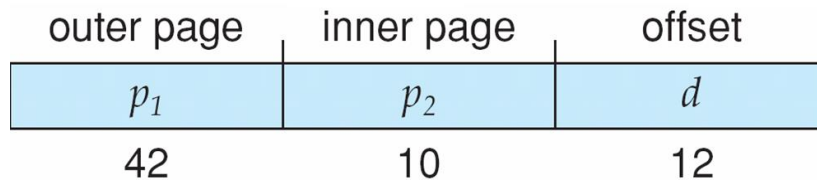


Pros/Cons of multi-level page tables

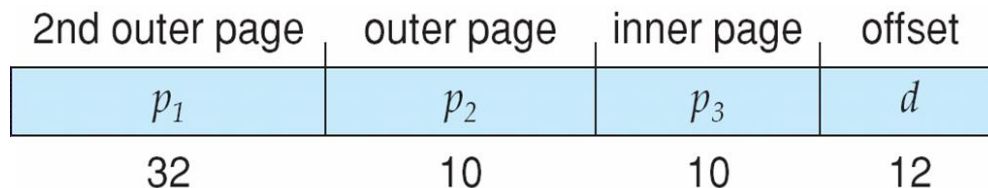
- Only allocates page-table space in proportion to the amount of address space you are using
 - compact and supports sparse address spaces
- if carefully constructed, each portion of the page table fits neatly within a page, making it easier to manage memory
 - For a large page table (say 4MB), finding such a large chunk of unused contiguous free physical memory can be quite a challenge. 
- On a TLB miss, two loads from memory will be required to get the right translation information from the page table
 - one for the page directory, and one for the PTE itself
 - time-space trade-off
- Complex

64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- **If page size is 4 KB (2^{12})**
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries, and outer page table has 2^{42} entries or 2^{44} bytes (=17TB)



- One solution is to add a 2nd outer page table
- But 2nd outer page table is still 2^{34} bytes (=17GB) in size
 - And possibly 4 memory access to get to one physical memory location



RISC-V page table

Register satp holds the physical address of the root page table address.
OS saves and rewrite satp when switching to another address space/application.

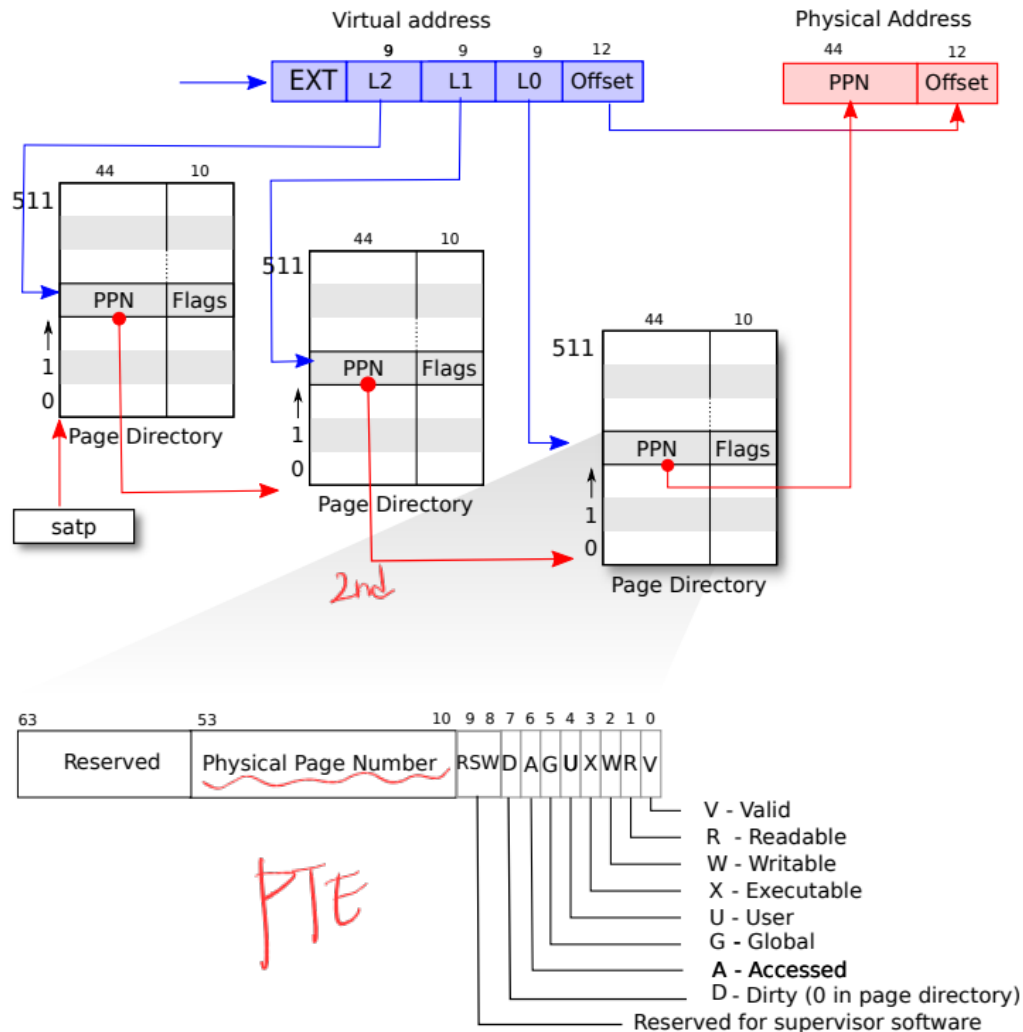
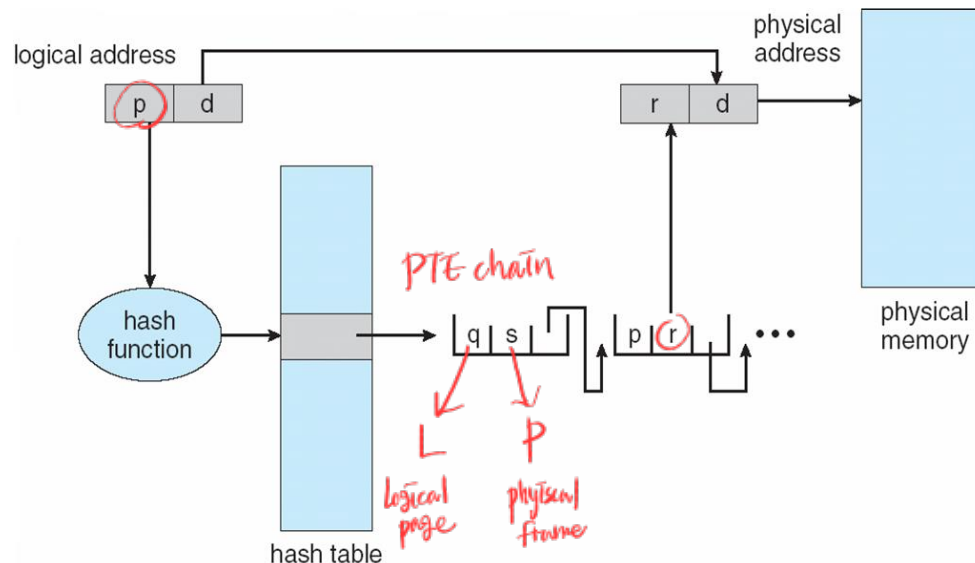


Figure 3.2: RISC-V page table hardware.

Hashed Page Tables

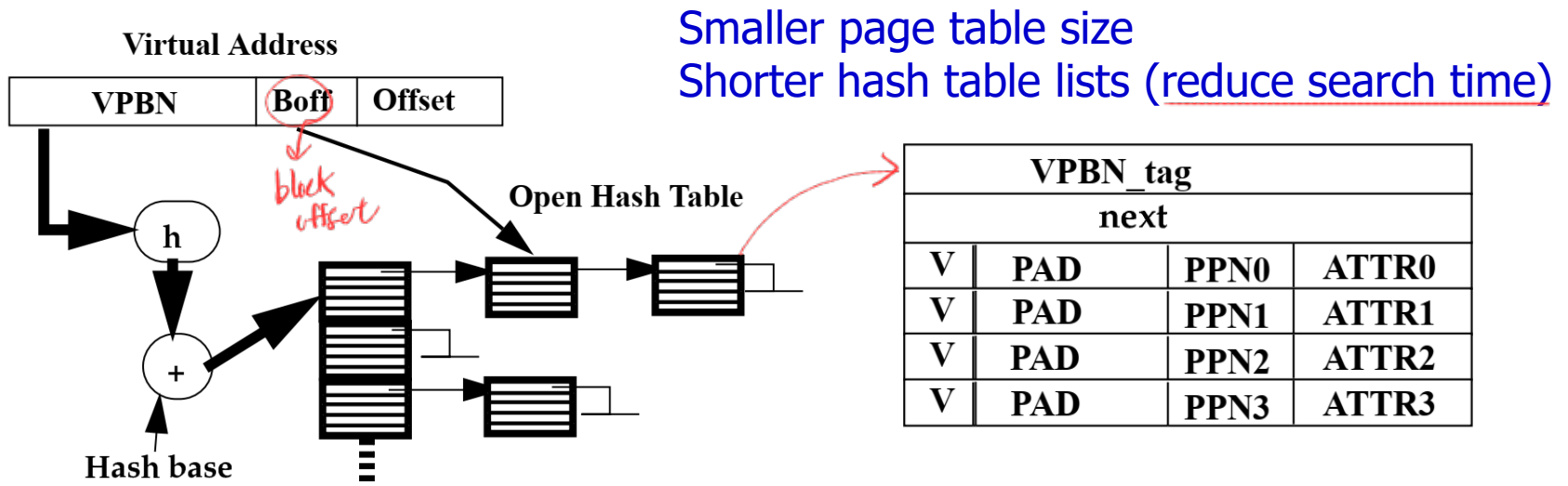
- Common approach for handling address spaces larger than 32 bits
- Logical page number is hashed into a page table
 - This page table contains a chain of elements hashing to the same location
- Each element contains
 - (1) the logical page number
 - (2) the value of the mapped page frame
 - (3) a pointer to the next element
- Logical page numbers are compared in this chain searching for a match
 - If a match is found, the corresponding physical frame is extracted



Hashed Page Table

- **Clustered page table**

- A variation of hashed page table for 64-bit addresses
- Similar to hashed but each entry refers to several consecutive virtual pages (such as 16) rather than 1
 - Single page table entry can store the mappings for several physical page frames
- Especially useful for bursty sparse address spaces (There are several scattered clusters.)

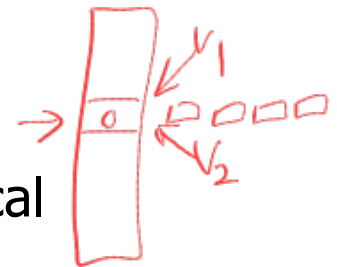


Inverted Page Table

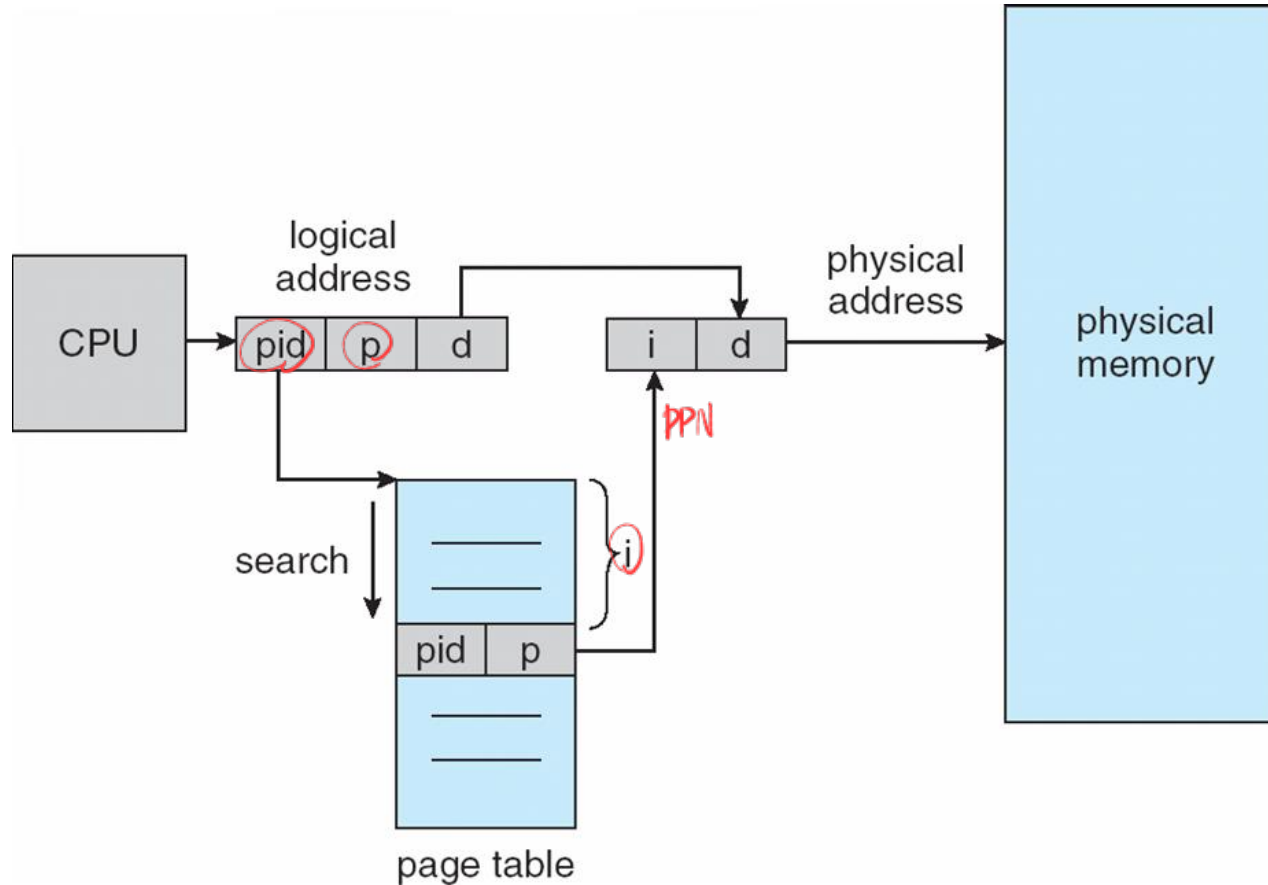
64bit



- PowerPC
- One entry for each real page of memory
 - Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
 - Since the table is shared, each entry must contain the process ID of the page owner.
 - Decreases memory needed to store each page table,
 - Increases time needed to search the table when a page reference occurs
 - Use hash table to limit the search to one or at most a few page-table entries
 - TLB can accelerate access
- But how to implement shared memory?
 - One mapping of a virtual address to the shared physical address



Inverted Page Table Architecture



Homework

- Homework in Chap 19 (TLB Measurement)

Quiz

1. In a 32-bit system with paging, the maximum size of the virtual address space is 1 GB. (X) *4GB*
2. A page fault must be preceded by a TLB miss. (O)
3. Hashed page tables are particularly useful for processes with sparse address spaces. (O)
4. An address generated by a CPU is referred to as a ~~physical~~ *logical* address in virtual memory system. (X)
5. A TLB miss could occur even though the requested page was in memory. (O)
6. An inverted page table has one page entry for each real page (or frame) of memory. (O)
7. If the invalid bit of a translation entry is set in TLB, a ~~page fault~~ *TLB miss* will occur. (X)
8. A large page size can increase the TLB reach. (O)
9. Multi-level page tables can ~~mitigate~~ *reduce* page fault handling overhead. (X) *오답 증가*
10. Multi-level page tables can reduce memory consumption for PMT. (O)
11. Each process in a modern OS typically has its own page table. (O)
12. The size of a page table increases with the physical memory size. (X) *physical X logical 관련*
13. If a page is not present in memory, the OS uses the page table to locate it on disk. (X) *PCB 참조.*
14. Even if the valid bit in a page table entry is not set, the corresponding page can be on disk. (X) *→ 사정키 안을 수 있음.*
15. The referenced (accessed) bit is used to track whether a page has been read or written recently. (O)
16. TLB entries are static and cannot be replaced once inserted. (X)
17. Without ASIDs, the TLB must be flushed on every context switch. (O)